MathViz 3D — Python Visualization Service Build Guide

> **Scope:** This document covers the design and implementation guidance for the Python Visualization Service — the orchestration layer that sits between the LLM Service and the Rust computation core. It receives validated Scene Descriptions, drives the Rust core via PyO3, manages step sequencing, and streams geometry bundles to the browser client over WebSocket.

---

1. Purpose and Role

The Visualization Service is the central coordinator of MathViz 3D's rendering pipeline. It owns no intelligence (that is the LLM Service's job) and does no heavy numerical work (that is the Rust core's job). Its job is to:

- Parse and validate incoming Scene Descriptions via Pydantic
- Route each step's geometry request to the correct concept handler
- Dispatch all computation to the Rust core in a single batch call
- Buffer step bundles in Redis
- Stream step bundles to the browser client over WebSocket
- Handle user playback events (step forward, back, jump, reset)

The service is intentionally stateless per-request. All session state lives in Redis. This allows horizontal scaling with no affinity requirements.

---

2. Service Structure

```
visualization_service/
├── main.py                  # FastAPI app factory, lifespan, router registration
├── config.py                # Settings via pydantic-settings (env vars)
├── dependencies.py          # FastAPI dependency injection (Redis, DB, auth)
│
├── api/
│   ├── routes.py            # HTTP and WebSocket route definitions
│   ├── websocket_manager.py # WebSocket connection lifecycle and message dispatch
│   └── auth.py              # Short-lived token validation for WebSocket upgrade
│
├── schema/
│   ├── scene_description.py # Pydantic v2 models for the full Scene Description
│   ├── step_descriptor.py   # Pydantic v2 model for StepDescriptor
```

```
│   ├── geometry_wire.py          # Pydantic v2 model for the step bundle wire format
│   └── enums.py                  # ConceptType, LayerMode, Transition, CoordinateSystem enums
│
├── sequencer/
│   ├── step_sequencer.py         # Core sequencer: batch dispatch, buffering, playback
│   ├── step_queue.py             # Ordered step queue with Redis-backed buffer
│   └── playback_events.py        # Event models: StepForward, StepBack, JumpTo, Reset
│
├── handlers/
│   ├── base.py                   # ConceptHandler abstract base class
│   ├── function_handler.py       # function_2d, function_3d, equation_buildup, series_convergence
│   ├── curve_handler.py          # parametric_curve_2d/3d, derivative_tangent
│   ├── surface_handler.py        # parametric_surface, implicit_surface
│   ├── vector_field_handler.py   # vector_field_2d, vector_field_3d
│   ├── ode_handler.py            # ode_phase_portrait, gradient_descent
│   ├── linalg_handler.py         # linear_transform, eigenspace_transform
│   ├── complex_handler.py        # complex_function
│   ├── topology_handler.py       # manifold
│   ├── statistical_handler.py    # distribution
│   └── numerical_handler.py      # riemann_sum, limit_approach
│
├── expression/
│   ├── parser.py                 # SymPy parse_expr + AST serialization
│   ├── symbols.py                # Symbol registry and constant substitution
│   ├── domain.py                 # DomainSpec → NumPy linspace arrays
│   └── cache_key.py              # (AST + domain) → deterministic hash string
│
├── geometry/
│   ├── bundle.py                 # StepBundle assembly: metadata + layer payloads
│   ├── serializer.py             # msgpack serialization of GeometryBuffer → wire format
│   └── delta.py                  # Delta geometry computation (new layers only)
│
├── cache/
│   ├── geometry_cache.py         # Redis geometry buffer cache (get/set/invalidate)
│   └── session_cache.py          # Redis session and step state cache
│
├── tasks/
│   ├── celery_app.py             # Celery app factory
│   └── export_tasks.py           # Background: PDF export, MP4 render, GIF generation
│
└── tests/
    ├── test_schema.py            # Pydantic model validation tests
    ├── test_handlers.py          # Handler unit tests with mocked Rust core
    ├── test_sequencer.py         # Step sequencer playback event tests
```

```
├── test_expression.py        # Expression parser + cache key tests
└── test_integration.py       # End-to-end: Scene Description → step bundles
```

---

3. Configuration (`config.py`)

All configuration is environment-variable driven via `pydantic-settings`. No secrets in source files.

Key settings groups:

Server: host, port, number of uvicorn workers, log level, CORS origins.

Redis: connection URL, geometry cache TTL (default 2 h), step bundle TTL (default 30 min), session TTL (default 24 h).

PostgreSQL: async connection URL, pool size, max overflow.

Rust Core: path to the compiled `mathviz_core` extension (for dev overrides; in production it is always on `sys.path`), max batch size per call.

Celery: broker URL (Redis), result backend URL, task time limits.

Security: WebSocket token signing secret, token TTL (60 s), rate limit thresholds per user tier.

Geometry: default surface density, max resolution cap (prevents clients from requesting 2048^2 surfaces), default step dwell ms (not enforced server-side, but sent to client as a hint).

Settings are loaded once at startup and injected via FastAPI's dependency system — never accessed as globals inside handlers.

---

4. API Layer (`api/`)

4.1 Routes (`routes.py`)

The service exposes two types of endpoints:

HTTP REST (authenticated via Bearer token):

`POST /scenes` — Accept a validated Scene Description JSON from the LLM Service. Trigger batch pre-computation. Return a `scene_id` and a WebSocket URL for the client to connect to.

`GET /scenes/{scene_id}/steps/{step_index}` — Retrieve a specific step bundle from Redis. Used by clients that missed a WebSocket frame or need to re-fetch a step after reconnection.

`GET /scenes/{scene_id}/state` — Return current playback state (current step, speed, is_playing).

`POST /scenes/{scene_id}/reset` — Reset the scene to step 1 and clear accumulated layer state.

`GET /health` — Liveness and readiness probe. Reports Redis connectivity and Rust core availability.

WebSocket:

`WS /scenes/{scene_id}/stream` — Primary streaming endpoint. Authenticated via a short-lived signed token passed as a query parameter. Once connected, the server streams step bundles and listens for playback event messages from the client.

4.2 WebSocket Manager (`websocket_manager.py`)

Manages the lifecycle of active WebSocket connections. Responsibilities:

- Accept and authenticate the WebSocket upgrade.
- Register the connection with the active `StepSequencer` for the scene.
- Start a background task that reads playback events from the client (step forward, back, jump, reset) and dispatches them to the sequencer.
- Forward step bundles from the sequencer's output queue to the WebSocket as binary frames.
- Handle disconnection cleanly: stop the sequencer's streaming, update Redis state, log the disconnect.

Message format from client to server: JSON-encoded `PlaybackEvent` (type + optional payload, e.g. `{"type": "jump", "step": 4}`).

Message format from server to client: msgpack binary frames containing serialized `StepBundle` objects.

Connection errors (client disconnects, network drops) must not crash the sequencer. The sequencer continues buffering steps in Redis so a reconnecting client can resume without recomputation.

4.3 Auth (`auth.py`)

WebSocket connections are authenticated via short-lived signed tokens issued by the API Gateway at session creation. The token contains: `user_id`, `session_id`, `scene_id`, `expires_at`. It is signed with HMAC-SHA256 using the service's token signing secret.

Validation: check signature, check expiry, check that `scene_id` in the token matches the URL parameter. Reject with 4008 (policy violation) on failure.

---

5. Schema Layer (`schema/`)

5.1 Scene Description Models (`scene_description.py`, `step_descriptor.py`)

All models use Pydantic v2 with strict validation. Key design decisions:

- Use `model_validator` (not `field_validator`) for cross-field validation (e.g., verify that `total_steps` matches `len(steps)`).
- Use `Annotated` types with `Field` constraints for numeric bounds (e.g., `steps: Annotated[int, Field(ge=64, le=2048)]`).
- All enum fields use Python `Enum` subclasses defined in `enums.py` — never raw strings inside models.
- The `expression` field on `StepDescriptor` accepts `str | list[str]` — a single expression or a list for multi-expression steps (e.g., a parametric curve with x(t) and y(t) as separate strings).
- `hud_equation` is always a plain LaTeX string — validated to be non-empty and under 500 characters.
- `annotations` is a list of `Annotation` sub-models with strict position and label validation.

Validation pipeline (called once, at scene ingestion):

1. `SceneDescription.model_validate(raw_json)` — structural and type validation.
2. Custom `model_validator` checks: step indices are contiguous from 1, `layer_mode` transitions are coherent (a `replace` after an `add` is valid; a `highlight` on step 1 is a warning).
3. Domain sanity: for every `DomainSpec` in the scene (global and per-step), assert min < max and clamp steps to [64, 2048].
4. Concept type check: assert every `concept_type` value is in the registered handler registry.

On validation failure, raise `SceneDescriptionValidationError` with structured details (which step, which field, what was wrong). The LLM Service catches this and may trigger a self-correction retry.

5.2 Geometry Wire Format (`geometry_wire.py`)

`StepBundle` model:

- `step_index: int`
- `step_label: str`
- `hud_equation: str` (LaTeX)
- `narration: str`
- `layer_mode: LayerMode`
- `transition: TransitionType`
- `layers: list[LayerPayload]`
- `annotations: list[Annotation]`
- `is_delta: bool`

`LayerPayload` model:

- `layer_id: str`
- `source_expression: str` (LaTeX, for display)
- `concept_type: ConceptType`
- `vertex_buffer: bytes` (msgpack-encoded f32 array)
- `normal_buffer: bytes` (msgpack-encoded f32 array, empty for curves)
- `index_buffer: bytes` (msgpack-encoded u32 array)
- `uv_buffer: bytes` (optional, empty if not applicable)
- `instance_buffer: bytes` (optional, for vector field arrows)
- `color_hint: str` (hex color, e.g. `"#4A90D9"`)
- `opacity: float`

The `StepBundle` is never stored as a Pydantic model in Redis — it is serialized directly to msgpack bytes by `geometry/serializer.py` before caching.

---

6. Expression Pipeline (`expression/`)

The expression pipeline is the bridge between the LLM's string expressions and the Rust core's AST input. It runs entirely in Python before any Rust call.

6.1 Parser (`parser.py`)

Entry point: `parse_expression(expr_str: str, allowed_symbols: set[str]) -> dict`

Steps:

1. **SymPy parse_expr:** Call `sympy.parse_expr(expr_str, transformations='all')`. This handles implicit multiplication, common function names, and standard infix notation. Wrap in a try/except and raise `ExpressionParseError` on failure with the original string and SymPy's error message.
2. **Symbol validation:** Extract all free symbols from the parsed expression. Check that every symbol is either in `allowed_symbols` (the declared variables for this step) or is a known named constant. Unknown symbols raise `ExpressionParseError` — this prevents expressions that reference undefined variables from silently evaluating to garbage.
3. **Constant substitution:** Replace all named constant symbols (pi, E, etc.) with their numeric `Float` values in the SymPy expression tree. Do this before AST serialization so the Rust core never needs a constant lookup table.
4. **AST serialization:** Convert the SymPy expression tree to a plain Python dictionary (the `ASTNode` dict format that Rust deserializes via serde_json). This is a recursive tree walk. Map SymPy node types to the

`ASTNode` variants defined in the Rust types. Key mappings:

- `sympy.Add` → `{"op": "add", "args": [...]}`
- `sympy.Mul` → `{"op": "mul", "args": [...]}`
- `sympy.Pow` → `{"op": "pow", "args": [base, exp]}`
- `sympy.sin`, `sympy.cos`, etc. → `{"op": "sin", "arg": ...}`
- `sympy.Symbol` → `{"op": "var", "name": "x"}`
- `sympy.Float` / `sympy.Integer` / `sympy.Rational` → `{"op": "lit", "value": 3.14159...}`

5. Return the AST dict. This dict is later JSON-serialized and passed to the Rust core.

6.2 Symbol Registry (`symbols.py`)

A module-level registry mapping:

- Standard variable names: `x`, `y`, `z`, `t`, `u`, `v`, `r`, `theta`, `phi`
- Named constants: `pi`, `e`, `tau`, `golden_ratio`, `euler_mascheroni`, `ln2`, `ln10`, `sqrt2` → their `f64` values
- User-defined parameters from the step's `ParameterMap`

The registry is rebuilt per-step from the step's domain and parameter declarations. It is not global mutable state.

6.3 Domain Builder (`domain.py`)

Entry point: `build_domain(domain_spec: DomainSpec) -> dict`

For each axis present in the `DomainSpec`, produce a NumPy linspace array: `np.linspace(min, max, steps)`. Return a dict mapping axis name to the array.

This dict is passed alongside the AST to the Rust batch evaluator as part of the `BatchRequest`. The Rust core uses it to know the grid shape and evaluation points.

For animated parameters (those with `{min, max, frames}` in the `ParameterMap`), generate a list of parameter value arrays — one per frame. The Rust core evaluates each frame independently; they are submitted as separate entries in the batch (with the frame index appended to the hash key).

6.4 Cache Key Generator (`cache_key.py`)

Entry point: `compute_cache_key(ast_dict: dict, domain_spec: DomainSpec) -> str`

Produce a deterministic string hash of the (AST, domain) pair. This is the key used in the Rust batch deduplication and in the Redis geometry cache.

Implementation: serialize the AST dict and domain spec to a canonicalized JSON string (keys sorted, floats rounded to 10 significant figures to avoid floating-point representation differences). Hash with SHA-256. Return the first 32 hex characters.

This key must be stable across Python restarts and across different processes — so avoid any hashing that uses Python's randomized hash seed (`hash()`).

---

7. Concept Handlers (`handlers/`)

7.1 Base Class (`base.py`)

```
class ConceptHandler(ABC):
    @abstractmethod
    def build_computation_spec(
        self,
        step: StepDescriptor,
        domain_arrays: dict,
        parsed_asts: list[dict],
    ) -> ComputationSpec:
        ...

    @abstractmethod
    def build_layer_metadata(
        self,
        step: StepDescriptor,
    ) -> LayerMetadata:
        ...
```

`ComputationSpec` is a dataclass describing which Rust function to call and with what arguments. It contains: `rust_function_name: str`, `request_payload: dict` (the JSON-serializable input for that Rust function).

`LayerMetadata` is a dataclass with: `layer_id: str`, `source_expression: str` (LaTeX), `concept_type: ConceptType`, `color_hint: str`, `opacity: float`, `render_hints: RenderHints`.

Handlers are stateless. They take a `StepDescriptor` and return specs. They never call the Rust core directly — that is the sequencer's job.

7.2 Handler Registry

A module-level dict maps each `ConceptType` enum value to its handler class instance. At service startup, all handlers are instantiated once and registered. Adding a new concept type requires only registering a new handler — no other code changes.

```
python
```

```
HANDLER_REGISTRY: dict[ConceptType, ConceptHandler] = {
    ConceptType.FUNCTION_2D: FunctionHandler(),
    ConceptType.FUNCTION_3D: FunctionHandler(),
    ConceptType.EQUATION_BUILDUP: FunctionHandler(),
    ConceptType.PARAMETRIC_CURVE_2D: CurveHandler(),
    ...
}
```

7.3 Handler Responsibilities by Type

FunctionHandler (`function_2d`, `function_3d`, `equation_buildup`, `series_convergence`):

- Parse the expression(s).
- For `equation_buildup`: the step's expression is the cumulative sum up to this step — the handler does not need to re-accumulate; it trusts the LLM to provide the correct cumulative expression.
- Build a `batch_evaluate` computation spec.
- Set `rust_function_name = "batch_evaluate"`.
- Metadata: choose color from a palette by step index.

CurveHandler (`parametric_curve_2d`, `parametric_curve_3d`, `derivative_tangent`):

- For `derivative_tangent`: parse both the base function and compute its derivative symbolically via SymPy before passing to Rust. The derivative is a second expression.
- Set `rust_function_name = "trace_curve"`.
- Include `include_arclength: true` in the payload so Rust returns the arc-length array for draw transitions.

SurfaceHandler (`parametric_surface`, `implicit_surface`):

- Determine explicit vs. implicit based on concept type.
- For implicit: ensure the domain has all three axes (x, y, z).
- Set `rust_function_name = "batch_evaluate"` for explicit; `"batch_evaluate_implicit"` for implicit.
- Apply surface density from render hints to determine the grid size.

VectorFieldHandler (`vector_field_2d`, `vector_field_3d`):

- Expect either two expressions (P, Q) for 2D or three (P, Q, R) for 3D.
- Set `rust_function_name = "process_vector_field"`.
- Include flags for whether to compute curl and divergence based on render hints.

ODEHandler (`ode_phase_portrait`, `gradient_descent`):

- For phase portraits: generate a grid of initial conditions from the domain spec.
- For gradient descent: a single initial condition from the step's parameter map.
- Set `rust_function_name = "solve_ode_batch"`.
- Wrap the IVP list in the expected `IVPSpec` format.

LinalgHandler (`linear_transform`, `eigenspace_transform`):

- Parse the matrix from the expression field (expected as a JSON-serializable nested list, e.g. `"[[1,2],[3,4]]"`).
- Set `rust_function_name = "visualize_linear_transform"`.
- For `eigenspace_transform`: request eigenvector and SVD layers via flags in the payload.

NumericalHandler (`riemann_sum`, `limit_approach`):

- For `riemann_sum`: extract subdivision count and partition method from the step's parameter map.
- Track `from_index` in session state so each step requests only new rectangles.
- Set `rust_function_name = "generate_riemann"`.
- For `limit_approach`: treat as a curve trace with an animated parameter approaching the limit point.

---

8. Step Sequencer (`sequencer/`)

The Step Sequencer is the most complex component in the Visualization Service. One instance exists per active scene session, managed by the WebSocket manager.

8.1 Lifecycle

1. **Created** when a validated Scene Description arrives at `POST /scenes`.
2. **Initialized:** builds the `StepQueue`, dispatches the batch computation call to Rust.
3. **Streaming:** after Step 1's bundle is ready, begins streaming to the connected WebSocket client.
4. **Event-driven:** listens for playback events and adjusts streaming accordingly.
5. **Destroyed:** when the WebSocket disconnects and the session TTL expires, or on explicit reset.

8.2 StepQueue (`step_queue.py`)

A simple ordered data structure holding one entry per step. Each entry tracks:

- `step_index: int`
- `computation_spec: ComputationSpec` (built by the handler)
- `layer_metadata: LayerMetadata`
- `step_descriptor: StepDescriptor` (original from the scene)
- `bundle_cache_key: str` (Redis key for the pre-computed bundle)
- `status: Literal["pending", "computing", "ready", "error"]`

The queue is ordered by `step_index`. The sequencer walks it in order, but supports random-access for jump events.

8.3 Batch Computation Dispatch

On initialization, the sequencer:

1. Iterates over all steps in the queue.
2. For each step: calls the appropriate handler's `build_computation_spec` and `build_layer_metadata`.
3. Collects all unique (expression, domain) pairs across all steps (deduplication by cache key).
4. Calls the Rust core once: `mathviz_core.batch_evaluate(batch_request_json)`.
5. Receives geometry buffers keyed by hash.
6. For each step: assembles the `StepBundle` from the geometry buffers + metadata.
7. Serializes each bundle to msgpack and stores in Redis with key `step_bundle:{scene_id}:{step_index}`.
8. Marks the step as `"ready"` in the queue.

Steps 4–8 run in a background asyncio task (using `asyncio.to_thread` to wrap the blocking Rust call). Step 1 is prioritized: the sequencer awaits only Step 1's bundle before beginning streaming; Steps 2+ are stored in Redis as they complete.

8.4 Streaming Logic

Once Step 1 is ready, the sequencer enters its streaming loop:

- If `is_playing`: wait for the current step's dwell time, then advance to the next step.
- On each step advance: read the bundle from Redis, send over WebSocket as a binary frame.
- If the next step is not yet ready (Redis miss): wait up to 200 ms for it to arrive. If it still is not ready, send a `{"type": "buffer", "step": N}` JSON control message to the client and wait longer.

The sequencer tracks the `active_step_index` and `accumulated_layer_ids` (all layers rendered so far in the session). This is needed for delta geometry computation.

8.5 Delta Geometry

When `layer_mode = "add"` on a step, the client expects only the new layer — not a full scene. The sequencer:

1. Compares the new step's layers against `accumulated_layer_ids`.
2. Sends only layers not already sent (new `layer_id` values).
3. Sets `is_delta = true` in the bundle.

When `layer_mode = "replace"`: send all layers, set `is_delta = false`, clear `accumulated_layer_ids`.

When `layer_mode = "highlight"`: send all layers with `is_delta = false`, but mark previous layers with `opacity` dimmed in the bundle metadata.

8.6 Playback Event Handling (`playback_events.py`)

Events received from the WebSocket client:

`StepForward`**:** Increment `active_step_index` by 1 (if not already at the last step). Fetch and send the next bundle.

`StepBack`**:** Decrement `active_step_index`. For `add`-mode steps, the client reconstructs the scene by re-applying all layers from step 1 to the target step. Send a full replay bundle (non-delta, containing all layers up to and including the target step).

`JumpTo(step_index)`**:** Set `active_step_index` to the requested step. Reconstruct the accumulated layer list up to that step. Send the appropriate bundle (full for `replace`/`highlight` modes; reconstructed for `add` mode).

`Reset`**:** Set `active_step_index = 1`. Clear `accumulated_layer_ids`. Send Step 1's bundle as a non-delta (full scene).

`SetSpeed(multiplier)`**:** Update the dwell time multiplier in Redis session state. No bundle change.

`SetPlaying(bool)`**:** Start or pause the auto-advance loop.

All events are validated against `PlaybackEvent` Pydantic models before processing. Invalid events are silently discarded (logged at WARNING level).

---

9. Geometry Assembly (`geometry/`)

9.1 Bundle Assembly (`bundle.py`)

Entry point: `assemble_bundle(step: StepDescriptor, rust_results: dict, layer_meta: list[LayerMetadata], is_delta: bool) -> StepBundle`

This function takes the raw geometry buffers returned by Rust (keyed by hash), the layer metadata from the handler, and the step descriptor, and assembles a `StepBundle` Pydantic model.

Steps:

1. For each layer in `layer_meta`, look up its geometry buffer from `rust_results` using the hash key.
2. If the hash key is missing (Rust returned an error for this entry), mark the layer as errored and include an error annotation at its expected position.
3. Assemble `LayerPayload` objects: combine the raw geometry buffer arrays with the layer metadata.
4. Set `is_delta` based on the sequencer's instruction.
5. Return the complete `StepBundle`.

9.2 Serializer (`serializer.py`)

Entry point: `serialize_bundle(bundle: StepBundle) -> bytes`

Converts a `StepBundle` to msgpack bytes for WebSocket transmission and Redis caching.

Design: do not serialize the Pydantic model to JSON first and then to msgpack — go directly from the model's fields to msgpack using `msgpack.packb`. This avoids the JSON intermediate and keeps the serialized size 30–50% smaller than JSON alone.

For array fields (`vertex_buffer`, `normal_buffer`, `index_buffer`): the Rust core returns these as NumPy arrays via PyO3. Convert to raw bytes with `.tobytes()` before packing — `msgpack` treats them as binary blobs. The client knows the dtype (f32 for vertices/normals, u32 for indices) from the wire format spec.

Include a version byte at the start of every bundle for forward compatibility.

9.3 Delta Computation (`delta.py`)

Entry point: `compute_delta_layers(all_layers: list[LayerPayload], accumulated_ids: set[str]) -> list[LayerPayload]`

Simply filters `all_layers` to those whose `layer_id` is not in `accumulated_ids`. Returns the filtered list. Updates the caller's `accumulated_ids` set as a side effect.

This is intentionally simple — the complexity of delta semantics is handled by the sequencer's layer-mode logic, not here.

---

10. Cache Layer (`cache/`)

10.1 Geometry Cache (`geometry_cache.py`)

Wraps Redis async operations for geometry buffer storage.

`async get_geometry(cache_key: str) -> bytes | None` Returns the raw msgpack bytes of a geometry buffer, or None on cache miss.

`async set_geometry(cache_key: str, data: bytes, ttl: int = 7200) -> None` Stores geometry bytes with a 2-hour TTL by default.

`async get_step_bundle(scene_id: str, step_index: int) -> bytes | None` Returns the pre-computed step bundle bytes for a specific step, or None.

`async set_step_bundle(scene_id: str, step_index: int, data: bytes, ttl: int = 1800) -> None` Stores a step bundle with a 30-minute TTL.

The geometry cache serves two purposes: within a call, it allows the Python layer to skip submitting an expression to Rust if a cached result already exists in Redis (from a prior session). The Rust core also performs within-call deduplication, but the Redis cache persists across calls and sessions.

Cache key format: `geo:{hash}` for geometry buffers; `step_bundle:{scene_id}:{step_index}` for step bundles.

10.2 Session Cache (`session_cache.py`)

`async get_scene_state(scene_id: str) -> SceneState | None` Returns the current playback state for a scene.

`async set_scene_state(scene_id: str, state: SceneState) -> None` Updates playback state (current step, speed, is_playing, accumulated layer IDs).

`async get_conversation_context(session_id: str) -> list[dict]` Returns the LLM conversation history for context-management purposes. (Read-only from the Visualization Service — the LLM Service writes this.)

`SceneState` is a dataclass: `scene_id`, `current_step`, `total_steps`, `is_playing`, `speed_multiplier`, `accumulated_layer_ids: list[str]`, `started_at`.

---

11. Background Tasks (`tasks/`)

11.1 Celery App (`celery_app.py`)

A standard Celery app using Redis as both broker and result backend. Task serializer is msgpack (matching the rest of the service). Time limits: soft 60 s, hard 120 s per task.

11.2 Export Tasks (`export_tasks.py`)

`export_pdf(scene_id: str, user_id: str)` Retrieves all step bundles from Redis for the scene. For each step, renders the 3D viewport to an offscreen buffer (using a headless Three.js via Node.js subprocess or a server-side SVG representation of the geometry). Assembles a PDF with one page per step: the 3D view, the HUD equation in LaTeX (rendered via WeasyPrint), and the narration text. Stores the PDF to object storage and emails a download link.

`export_video(scene_id: str, user_id: str, resolution: str)` Similar to PDF but outputs an MP4. Uses ffmpeg to encode the per-step frames. Each step is held for its dwell time; transitions are rendered as frame sequences.

Audio narration (if TTS is enabled) is muxed in.

`export_gif(scene_id: str, user_id: str)` Produces an animated GIF from the per-step frames. Lower resolution than MP4 (max 640px wide). Uses Pillow for GIF encoding.

Export tasks are triggered by HTTP POST to the API layer and return immediately with a task ID. The client polls `GET /tasks/{task_id}` for status. On completion, the download URL is stored in PostgreSQL against the user and scene.

---

12. Testing Strategy

12.1 Unit Tests

Schema tests (`test_schema.py`):

- Valid Scene Descriptions parse without error.
- Out-of-range step counts, missing required fields, and invalid enum values all produce `ValidationError`.
- Cross-field validation: `total_steps` mismatch, non-contiguous step indices, invalid `layer_mode` sequences.

Handler tests (`test_handlers.py`):

- Each handler produces a valid `ComputationSpec` for a known `StepDescriptor`.
- The `rust_function_name` matches the expected Rust function for each concept type.
- Handlers do not call the Rust core (mock `mathviz_core` at the import level).

Expression tests (`test_expression.py`):

- Valid expressions parse to correct AST dicts.
- Unknown symbols raise `ExpressionParseError`.
- Cache keys are stable (same input → same key across multiple calls).
- Named constants are resolved to numeric values in the AST.

Sequencer tests (`test_sequencer.py`):

- `StepForward` advances the step index correctly.
- `StepBack` reconstructs the correct layer list for add-mode steps.
- `JumpTo` assembles the correct accumulated layers.
- `Reset` clears state correctly.
- Delta computation returns only new layers.

12.2 Integration Tests (`test_integration.py`)

Use a real Redis instance (via `testcontainers`) and a mocked Rust core (returns pre-baked geometry buffers for known expressions).

Key scenarios:

- Submit a 5-step Scene Description → assert 5 step bundles appear in Redis with correct keys.
- Connect a mock WebSocket client → assert bundles arrive in order.
- Send `StepForward` events → assert the client receives the correct bundle for each step.
- Send `JumpTo(3)` → assert the client receives a reconstructed bundle for step 3 with all accumulated layers.
- Disconnect and reconnect → assert the client can resume from the last step via `GET /scenes/{id}/steps/{n}`.

12.3 Load Tests (Locust)

Defined in a separate `locustfile.py`. Simulates 50 concurrent users each:

1. POSTing a Scene Description (using a random preset from a fixed set of 10).
2. Connecting via WebSocket.
3. Advancing through all 7 steps at 1× speed.
4. Sending a `JumpTo(3)` event mid-playback.

Target p95: Step 1 bundle delivered within 150 ms of Scene Description acceptance. All subsequent steps delivered within 50 ms of the step event (Redis read).

---

13. Security

- **Expression inputs from LLM Service are re-validated** even though the LLM Service already validated them. The Visualization Service never trusts upstream validation.
- **SymPy `parse_expr` is called with `evaluate=False`** initially, then re-evaluated after symbol validation. This prevents SymPy from eagerly evaluating expressions that reference unknown symbols into garbage values.
- **No `eval()` or `exec()` anywhere** in the service. All dynamic dispatch is through the handler registry dict, not through runtime code evaluation.
- **Redis keys are namespaced and user-scoped.** The `scene_id` is a UUID generated server-side; clients cannot guess other users' scene keys.

- **WebSocket payloads are size-limited** (max 4 KB per message). Any message exceeding this is rejected and the connection is closed.
- **Celery tasks validate their inputs** before beginning work. A tampered task message cannot trigger export of another user's scene.

---

14. Extension Points

Adding a new Concept Handler:

1. Create a new handler class in `handlers/` implementing `ConceptHandler`.
2. Register it in the `HANDLER_REGISTRY` in `handlers/__init__.py`.
3. Add the corresponding `ConceptType` enum value in `schema/enums.py`.
4. No changes to the sequencer, route layer, or WebSocket manager are needed.

Adding a new playback event type:

1. Add a new variant to `PlaybackEvent` in `sequencer/playback_events.py`.
2. Add the handler case in `step_sequencer.py`'s event dispatch method.
3. No schema or API layer changes required.

Supporting a new LLM provider output format: The Visualization Service only consumes validated `SceneDescription` JSON — it is agnostic to the LLM provider. Changes to the LLM provider are entirely contained in the LLM Service.